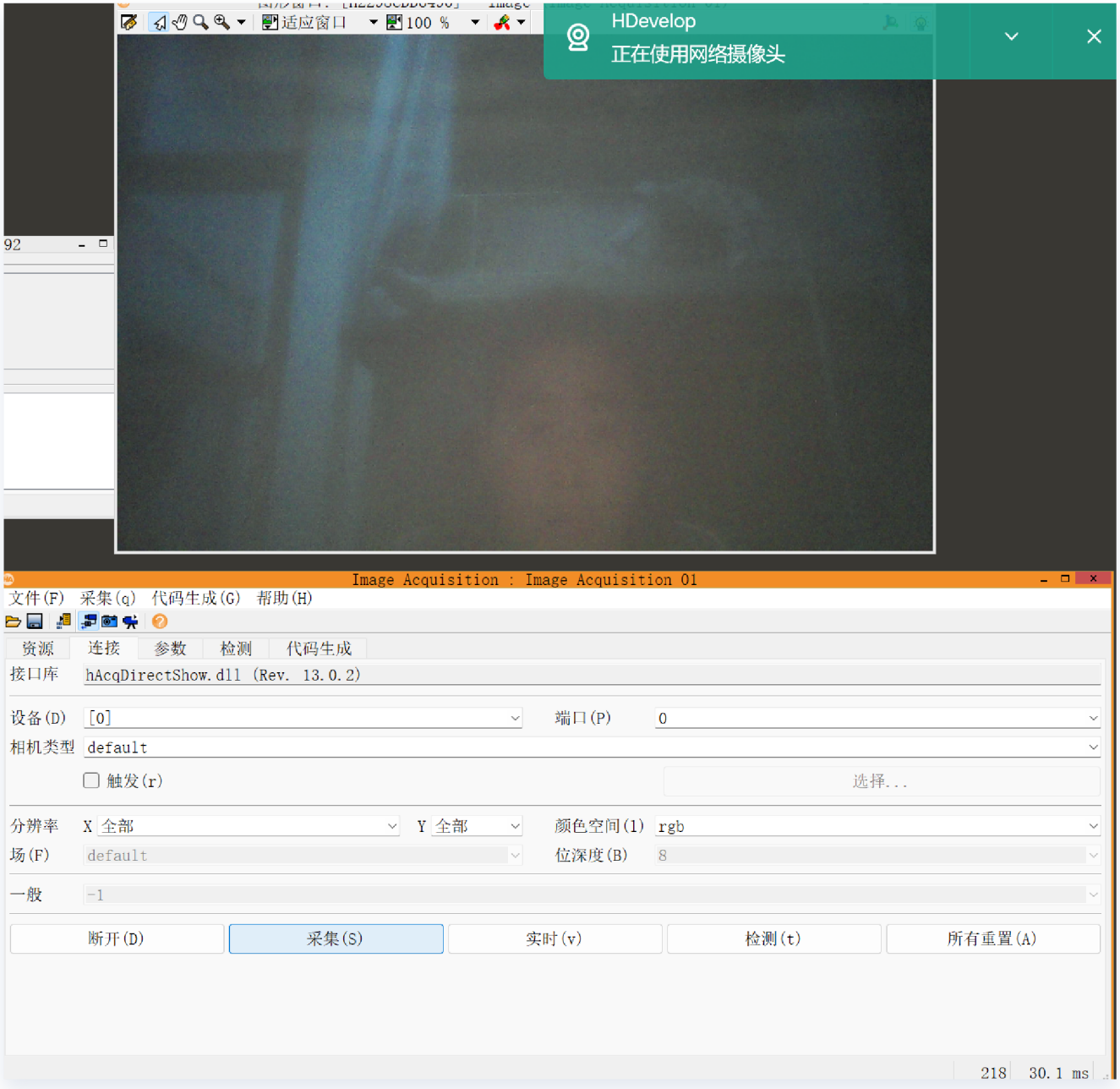
题目

编程题：利用Halcon的图像采集功能实现摄像头图像获取，尝试利用软触发模式采集图像，将程序导出为C/C++代码，并能成功运行程序。（无摄像头的同学可以读入本地视频实现）

效果展示



代码

```
* Image Acquisition 01: Code generated by Image Acquisition 01
open_framegrabber ('DirectShow', 1, 1, 0, 0, 0, 0, 'default', 8, 'rgb', -1,
'false', 'default', '[0] ', 0, -1, AcqHandle)
grab_image_start (AcqHandle, -1)
while (true)
    grab_image_async (Image, AcqHandle, -1)
    * Image Acquisition 01: Do something
endwhile
close_framegrabber (AcqHandle)
```

导出 C++ 代码

```
//////////////////////////////////////////////////////////////////////////////
//
// File generated by HDevelop for HALCON/C++ Version 18.11.0.1
// Non-ASCII strings in this file are encoded in local-8-bit encoding
(cp936).
// Ensure that the interface encoding is set to locale encoding by calling
// SetHcppInterfaceStringEncodingIsUtf8(false) at the beginning of the
program.
//
// Please note that non-ASCII characters in string constants are exported
// as octal codes in order to guarantee that the strings are correctly
// created on all systems, independent on any compiler settings.
//
// Source files with different encoding should not be mixed in one project.
//////////////////////////////////////////////////////////////////////////////
//



#ifndef __APPLE__
#  include "HalconCpp.h"
#  include "HDevThread.h"
#  if defined(__linux__) && (defined(__i386__) || defined(__x86_64__)) \
                         && !defined(NO_EXPORT_APP_MAIN)
#    include <X11/Xlib.h>
#  endif
#else
```

```
#  ifndef HC_LARGE_IMAGES
#    include <HALCONCpp/HalconCpp.h>
#    include <HALCONCpp/HDevThread.h>
#  else
#    include <HALCONCppxl/HalconCpp.h>
#    include <HALCONCppxl/HDevThread.h>
#  endif
#  include <stdio.h>
#  include <HALCON/HpThread.h>
#  include <CoreFoundation/CFRunLoop.h>
#endif



using namespace HalconCpp;


#ifndef NO_EXPORT_MAIN
// Main procedure
void action()
{

  // Local iconic variables
  HObject  ho_Image;

  // Local control variables
  HTuple  hv_AcqHandle;

  //Image Acquisition 01: Code generated by Image Acquisition 01
  OpenFramegrabber("DirectShow", 1, 1, 0, 0, 0, 0, "default", 8, "rgb", -1,
"false",
      "default", "[0] ", 0, -1, &hv_AcqHandle);
  GrabImageStart(hv_AcqHandle, -1);
  while (0 != 1)
  {
    GrabImageAsync(&ho_Image, hv_AcqHandle, -1);
    //Image Acquisition 01: Do something
  }
  CloseFramegrabber(hv_AcqHandle);

}


#ifndef NO_EXPORT_APP_MAIN
```

```
67
68  #ifdef __APPLE__
69  // On OS X systems, we must have a CFRunLoop running on the main thread in
70  // order for the HALCON graphics operators to work correctly, and run the
71  // action function in a separate thread. A CFRunLoopTimer is used to make
    sure
72  // the action function is not called before the CFRunLoop is running.
73  // Note that starting with macOS 10.12, the run loop may be stopped when a
74  // window is closed, so we need to put the call to CFRunLoopRun() into a loop
75  // of its own.
76  HTuple      gStartMutex;
77  H_pthread_t gActionThread;
78  HBOOL       gTerminate = FALSE;
79
80  static void timer_callback(CFRunLoopTimerRef timer, void *info)
81  {
82    UnlockMutex(gStartMutex);
83  }
84
85  static Herror apple_action(void **parameters)
86  {
87    // Wait until the timer has fired to start processing.
88    LockMutex(gStartMutex);
89    UnlockMutex(gStartMutex);
90
91    try
92    {
93      action();
94    }
95    catch (HException &exception)
96    {
97      fprintf(stderr,"  Error #%u in %s: %s\n", exception.ErrorCode(),
98              (const char *)exception.ProcName(),
99              (const char *)exception.ErrorMessage());
100   }
101
102   // Tell the main thread to terminate itself.
103   LockMutex(gStartMutex);
104   gTerminate = TRUE;
105   UnlockMutex(gStartMutex);
106   CFRunLoopStop(CFRunLoopGetMain());
107   return H_MSG_OK;
108 }
109
```

```
static int apple_main(int argc, char *argv[])
{
  Herror                error;
  CFRunLoopTimerRef     Timer;
  CFRunLoopTimerContext TimerContext = { 0, 0, 0, 0, 0 };

  CreateMutex("type","sleep",&gStartMutex);
  LockMutex(gStartMutex);

  error = HpThreadHandleAlloc(&gActionThread);
  if (H_MSG_OK != error)
  {
    fprintf(stderr,"HpThreadHandleAlloc failed: %d\n", error);
    exit(1);
  }

  error = HpThreadCreate(gActionThread,0,apple_action);
  if (H_MSG_OK != error)
  {
    fprintf(stderr,"HpThreadCreate failed: %d\n", error);
    exit(1);
  }

  Timer = CFRunLoopTimerCreate(kCFAllocatorDefault,
                               CFAbsoluteTimeGetCurrent(),0,0,0,
                               timer_callback,&TimerContext);
  if (!Timer)
  {
    fprintf(stderr,"CFRunLoopTimerCreate failed\n");
    exit(1);
  }
  CFRunLoopAddTimer(CFRunLoopGetCurrent(),Timer,kCFRunLoopCommonModes);

  for (;;)
  {
    HBOOL terminate;

    CFRunLoopRun();

    LockMutex(gStartMutex);
    terminate = gTerminate;
    UnlockMutex(gStartMutex);

    if (terminate)
```

```
      break;
    }

    CFRunLoopRemoveTimer(CFRunLoopGetCurrent(),Timer,kCFRunLoopCommonModes);
    CFRelease(Timer);

    error = HpThreadHandleFree(gActionThread);
    if (H_MSG_OK != error)
    {
      fprintf(stderr,"HpThreadHandleFree failed: %d\n", error);
      exit(1);
    }

    ClearMutex(gStartMutex);
    return 0;
  }
  #endif

  int main(int argc, char *argv[])
  {
    int ret = 0;

    try
    {
  #if defined(_WIN32)
      SetSystem("use_window_thread", "true");
  #elif defined(__linux__) && (defined(__i386__) || defined(__x86_64__))
      XInitThreads();
  #endif

      // file was stored with local-8-bit encoding
      //   -> set the interface encoding accordingly
      SetHcppInterfaceStringEncodingIsUtf8(false);

      // Default settings used in HDevelop (can be omitted)
      SetSystem("width", 512);
      SetSystem("height", 512);

  #ifndef __APPLE__
      action();
  #else
      ret = apple_main(argc,argv);
  #endif
    }
```

```
  catch (HException &exception)
  {
    fprintf(stderr,"  Error #%u in %s: %s\n", exception.ErrorCode(),
            (const char *)exception.ProcName(),
            (const char *)exception.ErrorMessage());
    ret = 1;
  }
  return ret;
}

#endif


#endif


```